

Leveraging Stable Diffusion for Style-Consistent Icon Generation of Kitchen Faucets

Alexander Morton
Georgia Institute of Technology
Master of Science in Analytics
Washington, DC
alexmorton@gatech.edu

I. ABSTRACT

The present investigation assessed the quality and consistency of generating kitchen faucet icons using stable diffusion. Initially, the attempt to create a singular text-to-image model (tuned using DreamBooth) to generate style-consistent icons with accurate attributes (number of handles, hole-fit type, etc.) proved unsuccessful despite extensive captioning efforts and prompting attempts. The base model ('stable-diffusion-v1-5'), tuned with Dreambooth, was unable to simultaneously learn accurate faucet features from real product images while training for a specific style (grey-scale with orange accents) based on existing icons provided by The Home Depot. However, tuning multiple models with unique types of faucet images for each while maintaining the same set of styling icons was more successful. This resulted in style-consistent icon generation for every type of kitchen faucet, but was computationally expensive and time consuming. Given the shortfall of not developing a singular text-to-image model that reaches desired results, further research is needed to find better suited stable diffusion models.

II. INTRODUCTION

Companies in the online retail space need to convey products in easy-to-comprehend ways for the consumer. Icons can help ensure product attributes are accurately conveyed, enhancing the ability for customers to navigate with limited frustration. In many cases, products have a variety of features and customizations that make detailed icon creation difficult. Icons of different types and styles are widely available already. However, licensing issues and finding style-consistent icons of related products make using these pre-made images largely impractical. Additionally, employing manual icon creation is time-consuming and lacks scalability. Thus, we need to develop a way to automatically generate icons. Artificial intelligence and existing deep learning practices offer potential ways to actualize this style-consistent icon automation process. With this investigation, we will explore stable diffusion (a generative artificial intelligence modeling process) and DreamBooth (a deep learning model-tuning process) to generate icons while evaluating their shortcomings and overall usability.

III. BACKGROUND

A. Objectives

Our main goals are to use deep learning and generative artificial intelligence (GenAI) to automate icon generation with regard to varying product attributes through text-to-image modeling. For this investigation, we will focus on generating style-consistent kitchen faucet icons with features that include number of handles, hole-fit type, neck type, and mount location. Other features are available for kitchen faucets (like color or finish family), but our chosen icon style that complies with The Home Depot's standard (grey-scale with orange accents) would not adequately reflect these traits. Additionally, we aim to develop a pipeline for creating uniform icons based on text data (captions), existing icons provided by The Home Depot, and existing product images from The Home Depot's website. On a more granular level, we will explore different model tuning processes and investigate their efficacy while performing trial-and-error parameter tuning. Additionally, we seek to develop a user-friendly pipeline that is easily accessible from beginning to end. Similarly, we aim to use resources that are publicly available while maintaining relatively low computational costs. We will also seek to incorporate tools with built-in free processing power (like Google Colab and Kaggle).

B. Stable Diffusion

Stable Diffusion^[3] is a generative AI model that supports text-to-image functionality and generates artwork based on user inputs. It takes a noisy (static-like) image and iteratively removes noise (static) to slowly build an image based on the given prompt using various predictions and neural networks. In our case, stable diffusion will provide the underlying process for generating images from text inputs. Many stable diffusion models are publicly available already, so we aim to find and use an existing model that integrates effectively within our pipeline. Some stable diffusion models require more computational resources or specialize in certain art styles, so choosing a model that meets our style needs at a low "cost" is important and may require some trial-and-error in addition to general research.

C. DreamBooth

DreamBooth^[1] is a deep learning system with text-to-image capabilities that allows users to fine-tune and create personalized stable diffusion models using their own sets of images and captions. Simply, DreamBooth is a training technique that alters an existing stable diffusion model based on new images and captions provided by the user. These images, in conjunction with DreamBooth, can be used to make a model learn a specific style (ex. a grey-scale 3D icon with orange accents) or unique subject (ex. a kitchen faucet with a spring-neck and two handles) that the original model may be unfamiliar with. For this analysis, we will deploy DreamBooth training through an existing Python-based Google Colab notebook ('fast-Dreambooth.ipynb'). The process of tuning a model with stable diffusion and DreamBooth is contained within this Jupyter notebook alongside the required dependencies and free (but limited) access to a high-performing remote GPU.

D. Literature Review

Online discourse surrounding stable diffusion and research efforts^[2] have grown significantly in the last two years. HuggingFace, an AI model storage community, has allowed for increased dissemination of stable diffusion models and their results. While official research into topics like stable diffusion and DreamBooth is still expanding, unofficial research in the form of trial-and-error has been conducted by the general AI community. From various forums and general sentiment analysis, we can gauge which base models are being used most, which are highly recommended, and usability of the more popular models. In general, most models heavily depend on training effort and the "best" models can suffer from high computational expenses. For this investigation, we will focus on user-friendly, publicly available, computationally inexpensive, and replicable methods.

IV. METHODOLOGY

A. Summary

Very few kitchen faucet or faucet-related images are available within the existing icon dataset provided by The Home Depot. Thus, we will develop a pipeline that fine-tunes an existing stable diffusion model using DreamBooth on Google Colab ('fast-Dreambooth.ipynb')^[6] using style-consistent icons and real product images from The Home Depot website^[7]. We will employ a general fine-tuning approach that incorporates style-consistent icons across various categories (not specific to kitchen faucets) and real product images (exclusively kitchen faucets) to create multiple text-to-image models that produce accurate kitchen faucet icons based on user prompts. After creating a range of models, we will store them on HuggingFace (a platform that stores AI models) and develop a user-friendly Jupyter notebook that accesses these models and outputs kitchen faucet icons based on text input and user choices.

B. Image Data Preprocessing

Before fine-tuning our models, data in the form of text-image pairings were created. For the styling portion, we selected a subset of nine (9) icons from the existing set of images provided by The Home Depot. In order to achieve style-consistent outputs that comply with the standards set by The Home Depot, the chosen subset only contains simple, three-dimensions (3D), grey-scale icons with occasional orange accents. Examples of these icons are proprietary of The Home Depot and sampled cannot be depicted

For the feature-tuning portion, a number of screenshots were captured from the kitchen faucet section of The Home Depot's website. These images were then used to train the model on different feature types (number of handles, hole fit, etc.). In total, thirty (30) distinct kitchen faucet images were collected to tune our text-to-image models. In order for our images to properly interact with the 'fast-Dreambooth.ipynb' DreamBooth tuning process, all images are resized using Birme^[5] to have dimensions of 512 x 512.

C. Text Data Preprocessing

With both the styling icons and faucet images, captions were constructed to describe the images in order to adequately tune our text-to-image models. These captions describe the image at a high level (ex. 'kitchen faucet') and a more detailed level (ex. 'two handles, chrome finish, ovalar deckplate') which hopefully enables our tuned models to identify and generate kitchen faucet icons with distinguishable features. Additionally, each caption contains a randomized key of characters (ex. 'abcdefghijkl') to further distinguish between the kitchen faucet images we provided and the idea of a kitchen faucet that the original model has already learned. In other words, adding these keys aims to increase the accuracy of output images in relation to the user's prompt by providing an extra layer of distinction between faucet images with different features. Captions for the styling icons all contain the same key ('thrdclrs') to help the model learn our desired style, while each faucet image receives a unique key. In order to pair our images and captions correctly and comply with the 'fast-Dreambooth.ipynb' requirements, images and text-files will both be named after their key. For example, a kitchen faucet image will be named 'abcdefghijkl (1).png' with its associated caption stored as 'abcdefghijkl (1).txt', whereas the training icons will be named 'thrdclrs (1).png', 'thrdclrs (2).png', and so forth to signify a grouping and enhance style-tuning. An example of a caption for a faucet image (with key 'faqtquyc') is shown below:



"faqtquyc, kitchen faucet with an arcing pipe and reflective chrome finish, two handles with one on the left and one on the right, faucet and handles sit on a small raised platform, blank white background"

D. Model Architecture

Our base model ('stable-diffusion-v1-5')^[8] is an existing text-to-image stable diffusion model that has been trained on thousands of images using loss functions and neural networks. Access to the untuned model can be found through HuggingFace at the path 'runwayml/stable-diffusion-v1-5'. This untuned model can also be referred to as our base model since we will build onto it using our own style icons and faucet images. After fine-tuning the base model using our final sets of images and captions, we will output our own models to HuggingFace, where we can store and access them in the future. Two methods of modeling are attempted. The first trains a single (1) model on all style icons and faucet images simultaneously. The second trains twelve (12) different models with each focusing on a distinct faucet type (ex. 'small standard with two handles', 'large spring-neck with one handle'). All twelve (12) models are tuned according to 1 – 4 distinct faucet images and the same subset of nine (9) styling images. Attempting both methods ideally provides a comparison for how accurate our tuned models can be when prompted with a detailed product description. This process may also give insight on the prevalence of over-fitting or under-fitting.

E. Fine-Tuning

The process of fine-tuning a stable diffusion model can be broken up into three (3) main components: the text encoder, U-Net image segmentation, and VAE (variational autoencoder). Simply, the text encoder turns input text (captions) into encoded data, the U-Net iteratively encodes and decodes data to denoise images based on predictions, and the VAE turns this final encoded data into images. Since our outputs are images (non-numeric), there are not analytical ways to efficiently "tune" the model since we cannot assess accuracy or error values. At most, we can evaluate runtimes to show computational effort. Additionally, due to the subjectivity of results, there could be multiple combinations of tuning parameter values that achieve an acceptable result. Thus, our methodology at the tuning level is largely based on trial-and-error. However, certain parameters do have recommended values to achieve different results. Within the 'fast-Dreambooth.ipynb' Google Colab notebook that we are using to tune our models, the following parameters along with their recommended values are described below:

- U-Net Training Steps: Each step is a batch of images and captions that the model uses to learn or update image characteristics. The recommendation is to start with 1500 or fewer steps and re-evaluate after observing results. For datasets of around 10 images, the default is set to 1500. We will tune models with 1500 and 500 steps to evaluate the trade-off between performance and computational effort.
- U-Net Learning Rate: Denotes how quickly the model adapts during tuning. A higher value means it may omit finer details in an effort to quicken the process. Given our outputs may require some finer details and the recommendation for sets of around 10 input images is 2e-6, we will use this default of 2e-6 as a constant for each model.

- Text Encoder Training Steps: Defaults to 350 steps. The recommendation is 200-450 steps in an effort to avoid overfitting. We will use this default of 350 as a constant for each model.
- Text Encoder Learning Rate: Defaults to a learning rate of 1e-6. The recommendation is to keep the rate low in an effort to avoid overfitting. We will use this default of 1e-6 as a constant for each model.
- Noise Offset: Denotes whether we would like to offset noise in the tuning process as a binary option (Yes/No). The recommendation is to offset noise if the model is being trained according to a style. Offsetting noise, in this case, exists to improve contrast and lighting-based factors within the image environment. We will tune models with and without the noise offset to evaluate its utility in creating simple icons.
- External Captions: Denotes whether the captions for the images come from a text file with the same name as the associated image. This is a binary option (Yes/No) that we will always select as 'Yes' since our captions are stored within text files.

F. Performance Evaluation

Since all outputs are images, our evaluation methods will be subjective. However, there are some visual checkmarks we can use to judge whether our generated icons are considered "good". First, we can assess whether our icons are in the same style as our training icons (similar color-scale and line-width). Second, we can identify disconnected lines or unrealistic attributes. This will vary based on the types of icons we are trying to generate, but in regard to kitchen faucets, these inconsistencies may include disconnect handles or multiple faucet heads. Lastly, if there is noise (unwanted background patterns, additional items beyond the kitchen faucet, etc.), we can consider this a "bad" output. This should be the easiest to identify. In this specific investigation, backgrounds should be completely white and the kitchen faucet icon should be the only visible design. Each person will have a different threshold for a "good" icon based on their needs. Examples of what we deem "good" or "bad" outputs will be evaluated later in the report.

G. Finalized Pipeline

The following is a simplified version of the developed pipeline: (i) The Home Depot provides reference icons to be used for styling; select a subset of these icons to create a style-consistent training set (ii) Capture products images from The Home Depot website to train our models on different kitchen faucet types (iii) Create captions within text files for each icon and product image (iv) Standardize image sizes using Birme^[5] to make all images and icons 512 x 512 (v) Use the Google Colab notebook 'fast-Dreambooth.ipynb' to tune and create new stable diffusion models; adjust tuning parameters as needed (vi) Upload new models to HuggingFace (vii) Use our developed Jupyter notebook 'IconGenerator.ipynb' to generate icons; this

self-developed code makes use of Python, PyTorch, Pandas, Python Imaging Library (PIL), matplotlib, ipywidgets, and diffusers (viii) Run our Jupyter notebook on Kaggle^[4] using its free GPU T4 X2 Accelerator or on your local machine (if sufficient resources are available).

V. RESULTS

A. Summary

The following results depict both “bad” icons (noisy, low-detail, or with general inconsistencies) and “good” icons (style-consistent and similar to training icons). These icons all resulted from one of the twelve distinct kitchen faucet models. The results shown are not the only icons generated from each model. We generated between 30-50 images for each model and chose outputs that best reflected the majority of icons produced for that specific model. All icons, regardless of parameter configuration, were prompted similarly by incorporating the words “thrdclrs style, simple sharp vectorized icon with black lines and light grey shading, blank white background” to capture our chosen style. However, it should be noted that the icons shown are subjectively selected and results may vary drastically based on the chosen prompt. Results are not shown from our large model that was tuned on all styling icons and faucet images simultaneously. In the case of this “large” model, results were shown to be drastically overfit despite extensive captioning and image variety. By overfit, we mean that the model would almost exclusively output one type of faucet (high-arching neck with one handle) even with training on multiple faucet types with easily distinguishable features. Although it created relatively style-consistent icons, it failed to generate kitchen faucets with unique attributes consistently for a text-to-image model.

B. Noisy Outputs

The following icons result from a model with 1500 U-Net Training Steps, 2e-6 U-Net Learning Rate, 350 Text Encoder Training Steps, 1e-6 Text Encoder Learning Rate, and no Noise Offset. Tuning this model took approximately 30 minutes when using the free T4 Accelerator within Google Colab. Each icon resulting from the model with these tuning parameters contained a lot of background noise and incorporated details that were not necessarily prompted. For example, the prompt used to generate these icons did not mention side sprayers, sinks, or background patterns that are apparent within these outputs. For reference, the model was tuned according to the nine style icons and two images of a double-handle, standard-neck kitchen faucet with a deckplate (see ‘faqtquyc’ faucet in ‘Text Data Preprocessing), neither of which contained any side items.



C. Low U-Net Training Step Outputs

The following icons result from a model with 500 U-Net Training Steps, 2e-6 U-Net Learning Rate, 350 Text Encoder Training Steps, 1e-6 Text Encoder Learning Rate, and Noise Offset. Tuning this model took approximately 15 minutes when using the free T4 Accelerator within Google Colab. Each icon resulting from the model with these tuning parameters contained less detail, but consistently maintained a white background and grey-scale color scheme. For example, each icon is missing distinguishable handles and lacks a fully identifiable faucet shape in some cases. For reference, the model was also tuned according to the nine style icons and two images of a double-handle, standard-neck kitchen faucet with a deckplate.



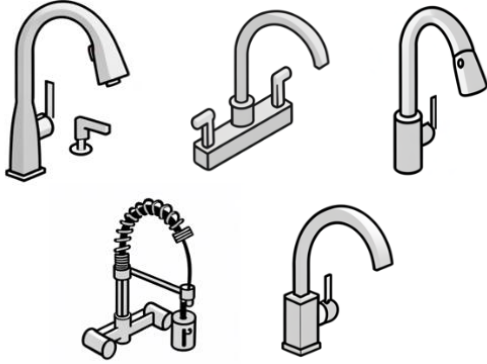
D. Bad Outputs

The following “bad” icons result from models with 1500 U-Net Training Steps, 2e-6 U-Net Learning Rate, 350 Text Encoder Training Steps, 1e-6 Text Encoder Learning Rate, and Noise Offset. These outputs come from the twelve kitchen faucet models that were individually trained on specific faucet types and were collected to exhibit common issues across all twelve models. The first icon (left) has its faucet neck disjoint from the base, making it unusable despite being style-consistent with the training icons. The second icon (middle) is unable to accurately depict a spring-neck due to the level of detail and non-uniformity of the spring. The third icon (right) has very high contrast and oversaturates the output with shadows due to reflections on the product in the original training image.



E. Good Outputs

The following “good” icons result from models with 1500 U-Net Training Steps, $2e-6$ U-Net Learning Rate, 350 Text Encoder Training Steps, $1e-6$ Text Encoder Learning Rate, and Noise Offset as well. These outputs come from the twelve kitchen faucet models that were individually trained on specific faucet types and were collected to exhibit the ability to achieve style-consistency across all twelve models. Tuning these twelve models took about 30 minutes each when using the free T4 Accelerator within Google Colab.



VI. CONCLUSION

A. Summary

Regarding our objectives, we successfully used deep learning (DreamBooth) and generative AI (stable diffusion) to create style-consistent kitchen faucet icons with different attributes using multiple text-to-image models. Additionally, we developed a user-friendly pipeline that incorporates publicly available environments (Google Colab) and model storage (HuggingFace) alongside a personally developed Jupyter Notebook that uses these models to generate kitchen faucet icons based on user selections. Based on the “good” icon results, we are able to semi-consistently generate grey-scale kitchen faucet images that comply with the Home Depot’s branding guidelines. Results can vary significantly between outputs based on the user’s prompt and can take up to (or beyond) 50 image generations to achieve an acceptable result. However, across all twelve models, we were able to generate at least one “successful” faucet icon for each. Certain faucet types were more difficult to achieve consistent outputs with than others. For example, spring-neck faucets (first icon on bottom row of “Good Outputs”) and 2-hole faucets (first icon on top row of “Good Outputs”) were less consistent. In general, the spring-necks were too non-linear and seemed to conflict with the simplicity of the styling icons. The 2-hole faucets faced difficulty generating two separate objects within the same icon output. Looking at our Low U-Net Training Step model and Noisy model, we can observe side objects being generated more easily, albeit not very detailed. We deemed that the benefits of maintaining a higher

number of U-Net training steps and continuously offsetting noise to create style-consistent icons outweighed the ability to consistently produce side objects at the expense of detail and clarity. For this reason, two of our twelve models were trained exclusively on soap dispensers and side sprayers so that these objects can be generated separately from the main kitchen faucet if needed. Wall-mounted faucets (right-most icon within “Bad Outputs”) were also difficult to generate since most (if not all) product images used for training were shown at an angle where the wall attachment portion was hidden behind the main body of the kitchen faucet. Thus, important aspects of the faucets structure are obscured from the model’s learning process and can explain poor results for wall-mounted kitchen faucets.

B. Recommendations

Based on trial-and-error, we were able to make certain observations regarding the efficacy of tuning stable diffusion models as it pertains to this specific pipeline. While using the ‘fast-Dreambooth.ipynb’ Google Colab notebook with the ‘stable-diffusion-v1-5’ model as the baseline, we developed several recommendations for crafting models in the future. First, we recommend training a model on at least 2 images of the same faucet type (same hole fit, number of handles, mount type, etc.) in addition to the styling icons. We found that using a single image was not sufficient for consistently creating accurate outputs as it did not give DreamBooth enough to learn from. Using two or more faucet training images results in purposeful overfitting and generates consistently accurate icons. This type of overfitting is welcome here since we create unique models for each faucet type, so generating one type of faucet icon allows us to focus our prompts on the style of the output rather than feature accuracy. Second, using product images with polished chrome or stainless steel finishes results in more accurate shading in the resulting icons. Since our original styling icons are generally more boxy with sharp edges, the shading does not usually translate to the softer edges and curves of kitchen faucets. Using training images with very clear shadows and shading in our desired grey color scheme (chrome and steel) does well to transfer these details onto our resulting icons. However, this can be overdone and you may get images like the third icon under ‘Bad Outputs’ where the shadows and reflections are overly contrasted. Thus, our recommendation is to use a mix of stainless steel and polished chrome faucet images to create balanced icons with accurate shading. Lastly, using our current method of captioning alongside the ‘stable-diffusion-v1-5’ model does not do well when tuning for both style and feature details at the same time. Thus, for now, we recommend creating individual models for each specific type of kitchen faucet in order to drastically increase accuracy of output icons.

C. Future Work

The major failing point of this investigation was not being able to create a singular text-to-image model that can distinguish between all different faucet features. Rather, we resorted to creating a series of models tuned individually to learn specific

kitchen faucet types. However, this does tell us that for future explorations the basic ‘stable-diffusion-v1-5’ model may not be robust enough to meet this need. There are more current and powerful stable diffusion models that can be investigated that may be able to handle simultaneous style-tuning and image-training at a higher level than we could achieve. This does not mean our exploration was useless, though. Our models and generated icons could be used as the basis for tuning future stable diffusion models given we generated multiple faucet icons of different types that are already style-consistent and thus more applicable than the images and icons used to tune our models. Many of the roadblocks faced in this investigation revolved around computational expense. Our chosen base model, ‘stable-diffusion-v1-5’, allowed us to tune tens of models before running out of free GPU resources, making it more accessible than other attempted models. Overall, this investigation achieved its goals and generated style-consistent kitchen faucet icons with the potential to build upon these outputs and develop a more singular model pipeline.

REFERENCES

- [1] Ruiz, Nataniel, et al. DreamBooth: Fine Tuning Text-To-Image Diffusion Models for Subject-Driven Generation. 25 Aug. 2022, <https://doi.org/10.48550/arxiv.2208.12242>.
- [2] Sohn, Kihyuk, et al. *StyleDrop: Text-To-Image Generation in Any Style*. 1 June 2023, <https://arxiv.org/pdf/2306.00983>
- [3] Rombach, Robin, et al. High-Resolution Image Synthesis with Latent Diffusion Models. ArXiv (Cornell University), 20 Dec. 2021, <https://doi.org/10.48550/arxiv.2112.10752>.
- [4] Kaggle. “Kaggle: Your Home for Data Science.” *Kaggle.com*, 2022, www.kaggle.com/.
- [5] “BIRME - Bulk Image Resizing Made Easy 2.0 (Online & Free).” *Www.birme.net*, www.birme.net/.
- [6] “Google Colab.” *Colab.research.google.com*, colab.research.google.com/github/TheLastBen/fast-stable-diffusion/blob/main/fast-DreamBooth.ipynb.
- [7] “Kitchen Faucets - the Home Depot.” *The Home Depot*, 2019, www.homedepot.com/b/Kitchen-Kitchen-Faucets/N-5yc1vZar0k.
- [8] “Runwayml/Stable-Diffusion-V1-5 · Hugging Face.” *Huggingface.co*, huggingface.co/runwayml/stable-diffusion-v1-5.